dr. sc. Sebastijan Horvat
Helena Marciuš, mag. inf. et math.
Luka Rak, mag. math.

Uvod u programiranje

Zbirka zadataka

Zagreb
13. studenoga 2026.

Sadržaj

1 Uvod **1**

2 Varijable **4**

3 Osnovne aritmetičke operacije **6**

4 Grananje **11**

5 Petlje **20**

6 Naredba `switch case` **29**

7 Razni zadaci **33**

8 Nizovi **49**

Poglavlje 1

Uvod

Primjer 1.1. Program koji ispisuje tekst

"To C or not to C: that is the question".

```c
#include <stdio.h>

int main(void) {
    printf("To C or not to C: that is the question");
    return 0;
}
```

Primjer 1.2. Proučimo greške koje se javljaju u sljedećem kodu.

```c
        #include <stdio.h>

        int main(void){
            print("Hello, world);
            return 0
        }
```

Kompajler javlja sljedeće greške:

```
try.c:5:13: error: implicit declaration of function ‘print’; did you
↪  mean ‘printf’? [-Wimplicit-function-declaration]
    5 |             print("Hello, world);
```

```
      |             ^~~~~
      |             printf
try.c:5:19: warning: missing terminating " character
    5 |             print("Hello, world);
      |                   ^
try.c:5:19: error: missing terminating " character
    5 |             print("Hello, world);
      |                   ^~~~~~~~~~~~~~~
try.c:6:13: error: expected expression before ‘return’
    6 |             return 0
      |             ^~~~~~
try.c:6:21: error: expected ‘;’ before ‘}’ token
    6 |             return 0
      |                     ^
      |                     ;
    7 |         }
      |         ~
```

Naziv funkcije je `printf`, a ne `print`. Tekst koji proslijeđujemo kao argument funkciji `printf` treba započeti i završiti sa znakom `"`. Naredbe završavaju znakom `;`.

Primjer 1.3. Promotrimo sljedeći kod.

```c
#include <stdio.h>

int main(void){
    printf("Hello");
    printf("world");
    return 0;
}
```

Program ispisuje:

```
Helloworld
```

Funkcija `printf` nastavlja pisati gdje je kursor stao. Ako želimo pisati u novi red, moramo koristiti `\n`.

```c
#include <stdio.h>

int main(void) {
    printf("Hello\nworld");
    return 0;
}
```

Ispis:

```
Hello
world
```

Zadatak 1.4. Napišite program koji ispisuje sljedeću poruku:

`\n unutar "a\na" je za prelazak u novi red.`

Rješenje.

```c
#include <stdio.h>

int main(void) {
    printf("\\n unutar \"a\\na\" je za prelazak u novi red.");
    return 0;
}
```

Zadatak 1.5. Istražite čemu služe `\t`, `\r` i `\a`.

Poglavlje 2

Varijable

Primjer 2.1. Program koji učitava dva cijela broja te ispisuje njihove vrijednosti.

```c
#include <stdio.h>

int main(void) {
    int a;
    int b;
    scanf("%d %d", &a, &b);
    printf("%d %d", a, b);
    return 0;
}
```

Deklaraciju varijabli istog tipa možemo napraviti u jednoj liniji s `int a, b;`. Brojeve možemo učitati i naredbom `scanf("%d%d", &a, &b);` (bez razmaka). To je zato jer drugi specifikator `%d` ignorira sve bjeline (ENTER) prije drugog broja. Općenito, to nije slučaj (npr. za `%c`).

Primjer 2.2. Promotrimo vrijednost varijable `a` tijekom izvođenja sljedećeg programa. Pretpostavimo da je korisnik učitao vrijednost 3.

```c
#include <stdio.h>

int main(void) {
    int a = 1; // inicijalizacija
```

```
5       printf("a = %d, &a = %p\n", a, (void*)&a);
6       a = 2;
7       printf("a = %d, &a = %p\n", a, (void*)&a);
8       scanf("%d", &a); // ucitamo 3
9       printf("a = %d, &a = %p\n", a, (void*)&a);
10      return 0;
11  }
```

U terminalnu dobivamo sljedeće:

```
1  a = 1, &a = 0x7ffc2a4ba78c
2  a = 2, &a = 0x7ffc2a4ba78c
3  3
4  a = 3, &a = 0x7ffc2a4ba78c
```

Uočimo da se vrijednost varijable `a` mijenja, no adresa ostaje ista.

Poglavlje 3

Osnovne aritmetičke operacije

Zadatak 3.1. Napišite program koji učitava dva cijela broja te ispisuje njihov zbroj i razliku u decimalnom, oktalnom i heksadecimalnom zapisu. Rezultate ispišite tablično tako da svaki ispisani rezultat zauzima najmanje 10 mjesta. Pretpostavljamo da su brojevi takvi da su im zbroj i razlika nenegativni cijeli brojevi.

Rješenje. Format za ispis oktalnog zapisa broja je `%o`, a format za ispis heksadecimalnog zapisa broja je `%x`.

```c
#include <stdio.h>

int main(void) {
    int a, b;
    int zbroj, razlika;
    printf("Unesite dva broja: ");
    scanf("%d %d", &a, &b);
    zbroj = a + b;
    razlika = a - b;
    printf("%-10s %10s %10s %10s\n",
        "Rezultat", "Decimalno", "Oktalno", "Heks.");
    printf("%-10s %10d %10o %10x\n",
        "Zbroj", zbroj, zbroj, zbroj);
    printf("%-10s %10d %10o %10x\n",
        "Razlika", razlika, razlika, razlika);
    return 0;
}
```

Zadatak 3.2. Napišite program koji učitava početnu cijenu proizvoda u eurima i postotak popusta. Program treba izračunati iznos popusta te konačnu cijenu proizvoda nakon popusta. Sve rezultate ispišite na dvije decimale. Kod ispisa postotka potrebno je ispisati i znak %.

Rješenje. Zaokruživanje realnog broja na dvije decimale vrši se dodavanjem `.2` u oznaku formata ispisa. Općenito, zaokruživanje na `n` decimala vrši se dodavanjem `.n` u oznaku formata ispisa.

```c
#include <stdio.h>

int main(void) {
    double cijena, postotak;
    double popust, konacna_cijena;

    printf("Unesite cijenu proizvoda: ");
    scanf("%lf", &cijena);
    printf("Unesite postotak popusta: ");
    scanf("%lf", &postotak);

    popust = cijena * postotak / 100;
    konacna_cijena = cijena - popust;

    printf("Pocetna cijena: %.2f EUR\n", cijena);
    printf("Popust: %.2f%% (%.2f EUR)\n", postotak, popust);
    printf("Konacna cijena: %.2f EUR\n", konacna_cijena);

    return 0;
}
```

Zadatak 3.3. Napišite program koji učitava realne brojeve `x` i `y` te ispisuje vrijednost polinoma

$$p(x, y) = x^3y - 2xy^2 + 1.$$

Rješenje.

```c
#include <stdio.h>

int main(void){
```

```
4       double x, y, p;
5       scanf("%lf %lf", &x, &y);
6       p = x*x*x*y - 2*x*y*y + 1;
7       printf("%f", p);
8       return 0;
9   }
```

Alternativno, možemo izbjeći korištenje dodatne varijable `p`:

```
1   double x, y;
2   scanf("%lf %lf", &x, &y);
3   printf("%f", x*x*x*y - 2*x*y*y + 1;);
```

Zadatak 3.4. Napišite program koji učitava trajanje filma izraženo u sekundama. Program treba izračunati i ispisati koliko to trajanje iznosi u satima, minutama i sekundama.

Rješenje.

```
1   #include <stdio.h>
2
3   int main(void) {
4       int sekunde, sati, minute;
5
6       printf("Unesite broj sekundi: ");
7       scanf("%d", &sekunde);
8
9       sati = sekunde / 3600;
10      sekunde = sekunde % 3600;
11      minute = sekunde / 60;
12      sekunde = sekunde % 60;
13
14      printf("%d h, %d min i %d sek\n", sati, minute, sekunde);
15
16      return 0;
17  }
```

Budući da su varijable tipa `int`, u linijama 9 i 11 radi se o cjelobrojnom dijeljenju, što i želimo. Operator `%` daje ostatak pri djeljenju.

Izrazi oblika `x = x OP y;` gdje je `OP`=$\{$`+`, `-`, `*`, `%`, `/`$\}$, mogu se skraćeno pisati `x OP= y;`:

```
sati = sekunde / 3600;
sekunde %= 3600;
minute = sekunde / 60;
sekunde %= 60;
```

Zadatak 3.5. Napišite program koji od korisnika traži unos polumjera kugle r izraženog u metrima. Program treba izračunati obujam kugle prema formuli

$$V = \frac{4}{3}\pi r^3.$$

Dobiveni rezultat ispišite u kubnim metrima koristeći znanstvenu notaciju (e-notaciju) s tri decimale.

Rješenje. Format za ispis realnog broja u e-notaciji je `%e`, odnosno `%E`, ovisno želimo li malo slovo e ili veliko slovo E.

```
#include <stdio.h>

int main(void) {
    double r, V, pi = 3.14159; // moze i 3.14 :)

    printf("Unesi polumjer kugle u metrima: ");
    scanf("%lf", &r);

    V = (4.0 / 3.0) * pi * r * r * r;   // moze 4.0/3, ali ne 4/3
    printf("Obujam kugle je: %.3e m^3\n", V);
    // ili %.3E za veliko E u e-notaciji

    return 0;
}
```

Napomenimo da u liniji 9 ne bi bilo ispravno napisati `4/3` jer je to cjelobrojno dijeljenje čiji rezultat je 1. Umjesto `4.0/3.0` mogli smo napisati i `4.0/3`.

Zadatak 3.6. Napišite program koji učitava početnu cijenu proizvoda u eurima, količinu naručenih proizvoda, iznos popusta u postocima i troškove

dostave. Program treba izračunati iznos konačne cijene narudžbe (s uračunatom količinom proizvoda, popustom i troškovima dostave) te ju ispisati. Koristite operatore složenog pridruživanja.

Rješenje.

```c
#include <stdio.h>

int main(void){
    double cijena, popust, dostava;
    int kolicina;

    printf("Unesite redom cijenu proizvoda, kolicinu narucenih
    ↪  proizvoda, popust te trosak dostave: ");
    scanf("%lf %d %lf %lf", &cijena, &kolicina, &popust, &dostava);
    cijena *= (1-popust/100); // cijena s popustom
    cijena *= kolicina; // cijena svih proizvoda zajedno
    cijena += dostava; // cijena s dodanim troskom dostave
    printf("Ukupna cijena je %.2lf\n", cijena);
    return 0;
}
```

Zadatak 3.7. Napišite program koji učitava tri novčana iznosa izražena u hrvatskim kunama (HRK) te ih pretvara u eure (EUR). Koristite fiksni tečaj konverzije: 1 EUR = 7.5345 HRK.
Iznose ispišite tablično, pri čemu svaki iznos treba zauzimati najmanje 10 mjesta. Iznose pri ispisu prikažite s dvije decimale i poravnajte ih lijevo unutar pripadajućeg stupca. Tablica treba sadržavati horizontalne (-) i vertikalne linije (|). U cijelom programu smijete samo jednu varijablu tipa double (ne računajući varijablu za iznos fiksnog tečaja konverzije).

Primjer ispisa:

```
+------------+------------+
|HRK         |EUR         |
+------------+------------+
|100.00      |13.27       |
|150.69      |20.00       |
|930.13      |123.45      |
+------------+------------+
```

Poglavlje 4

Grananje

Primjer 4.1. Program koji učitava dva cijela broja `a` i `b` te ispisuje rezultat sljedećih usporedba:

- `a < b`
- `a >= b-2`
- `a+6 != b-2`
- `a%b == 0`

```c
#include <stdio.h>

int main(void){
    int a, b;
    printf("Unesite dva cijela broja: ");
    scanf("%d %d", &a, &b);
    printf("%d < %d je %d\n", a, b, a < b);
    printf("%d >= %d-2 je %d\n", a, b, a >= b-2);
    printf("%d+6 != %d-2 je %d\n", a, b, a+6 != b-2);
    printf("%d%%%d == 0 je %d\n", a, b, a%b == 0);
    return 0;
}
```

Primjer 4.2. Program koji učitava tri cijela broja `a`, `b` i `c` te ispisuje vrijednost sljedećih izraza:

- `!(a && b) || c`
- `(!a || !b) == !(a && b)`
- `(a <= c) && !(b > c)`

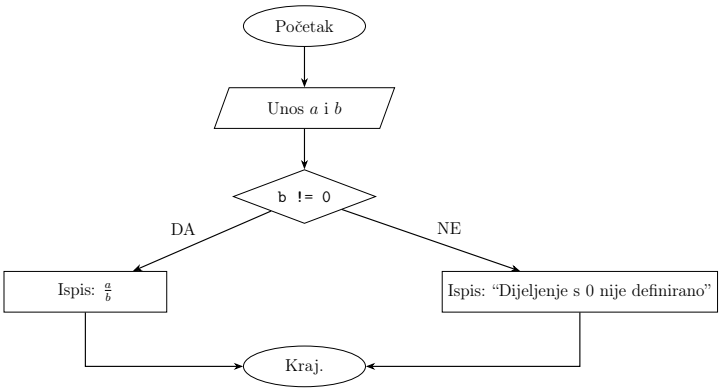
```c
#include <stdio.h>

int main(void){
    int a, b, c;
    printf("Unesite tri cijela broja: ");
    scanf("%d %d %d", &a, &b, &c);
    printf("!(%d && %d) || %d je %d\n", a, b, c, !(a && b) || c);
    printf("(!%d || !%d) == !(%d && %d) je %d\n", a, b, a, b, (!a ||
    ↪  !b) == !(a && b));
    printf("(%d <= %d) && !(%d > %d) je %d\n", a, c, b, c, (a <= c)
    ↪  && !(b > c));
    return 0;}
```

Budući da C sve vrijednosti različite od 0 smatra istinom, uneseni brojevi `a` i `b` `c` mogu biti bilo koji cijeli brojevi, ne samo 0 i 1.

Primjer 4.3. Program koji učitava dva cijela broja `a` i `b` te ispisuje njihov količnik $\frac{a}{b}$, odnosno odgovarajuću poruku ako količnik nije moguće izračunati. Količnik nije moguće izračunati ako je vrijednost varijable `b` jednaka 0.

Program možemo prikazati sljedećim dijagramom odlučivanja:

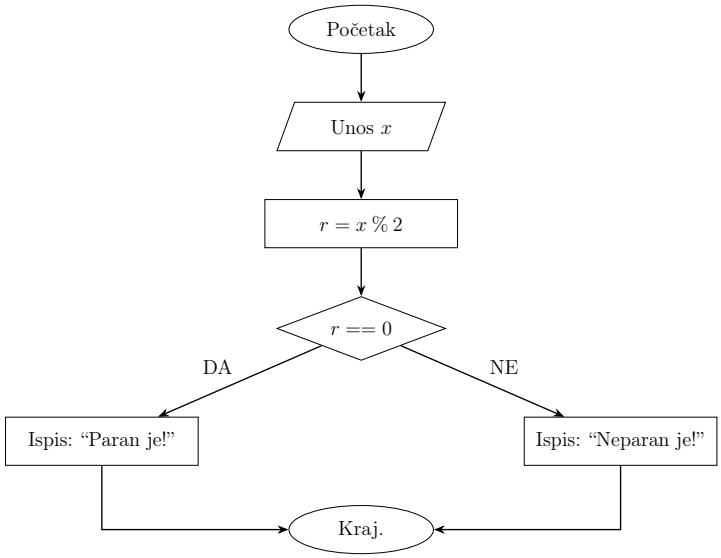


```
1  #include <stdio.h>
2
3  int main(void){
4      double a, b;
5      printf("Unesite dva realna broja: ");
6      scanf("%lf %lf", &a, &b);
7      if(b != 0)
8          printf("%.2f/%.2f = %.2f\n", a, b, a/b);
9      else
10         printf("Dijeljenje s 0 nije definirano!\n");
11     return 0;
12 }
```

Napomenimo da ne bi bilo ispravno staviti znak `;` (točka-zarez) nakon `if(b != 0)` (u liniji 7). Naime, znak `;` označio bi kraj `if` bloka te se u slučaju da je uvjet `b != 0` točan ne bi dogodilo ništa (kao da smo napisali „ako je `b` različito od 0, ne radi ništa"). Tada se linija 8 ne bi smatrala dijelom `if` bloka, nego bi se ispisivala neovisno o unesenim brojevima. Također, kompajler bi u tom slučaju javio `else without prevous if`.

Zadatak 4.4. Napišite program koji provjerava je li uneseni broj paran i ispisuje pripadnu poruku.

Rješenje. Dijagram odlučivanja za zadani program je sljedeći:



```
1   #include <stdio.h>
2
3   int main(void){
4       int x, r;
5       printf("Unesite realan broj: ");
6       scanf("%d", &x);
7       r = x%2;
8       if(r == 0)
9           printf("Paran je!\n");
10      else
11          printf("Neparan je!\n");
12      printf("Kraj.\n");
13      return 0;
14  }
```

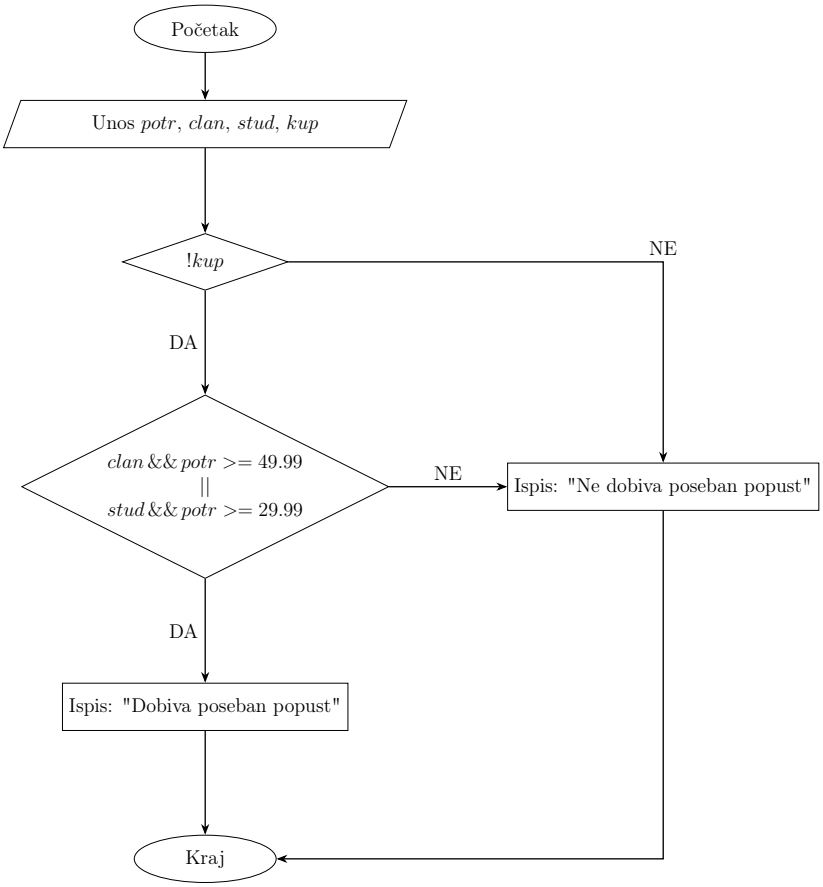
Da smo u liniji 8 napisali `if(r = 0)` (umjesto `if(r == 0)`), to ne bi bilo provjera jednakosti nego pridruživanje vrijednosti. Tada varijabla `r` postaje 0. No ono što je važnije, izraz `r = 0` vraća vrijednost pridruživanja (a to je 0). Stoga bi onda uvijek bilo `if(0)`, odnosno program bi uvijek ispisao `"Neparan je"`, neovisno je li uneseni broj paran ili neparan.

Napomenimo da smo u liniji 8 mogli imati `if(x%2 == 0)` umjesto da vrijednost `x%2` spremamo u zasebnu varijablu.

Zadatak 4.5. Kupac dobiva poseban popust ako je učlanjen i potrošio je barem 49.99 €, ili ako je student i potrošio je barem 29.99 €. Međutim, ako već koristi nekakav kupon za popust, ne može dobiti posebni popust. Napišite program koji za zadane podatke ispisuje dobiva li kupac poseban popust ili ne.

Rješenje.

Kada bismo tekst zadatka doslovno preveli u dijagram odlučivanja, on bi bio sljedeći:



Međutim, u kodu možemo napisati jedan „složeni" uvjet koji obuhvaća sve uvjete navedene prethodnim dijagramom:

```c
#include <stdio.h>

int main(void){
    int clan, stud, kup;
    double potr;
    printf("Potrosio? Uclanjen? Student? Kupon?");
    scanf("%lf %d %d %d", &potr, &clan, &stud, &kup);
    if( ((clan && potr >= 49.99) || (stud && potr >= 29.99))
        && !kup)
            printf("Dobiva poseban popust");
    else
        printf("Ne dobiva poseban popust");
```

```
13      return 0;
14  }
```

Zadatak 4.6. Najveća dopuštena težina kofera u avionu je 20 kg. Napišite program koji učitava težinu (u kg) kofera putnika. Ako je težina manja ili jednaka dozvoljenoj, program ispisuje poruku `Sretan put`. U suprotnom, program učitava koliko kilograma putnik želi izvaditi iz kofera te ispisuje novu težinu kofera. Ako je nova težina manja ili jednaka dozvoljenoj, program ispisuje poruku `Sretan put`, inače ispisuje poruku `Kofer je i dalje pretezak!`

Rješenje.

```c
#include <stdio.h>

int main(void){
    double tezina;
    printf("Unesite tezinu kofera: ");
    scanf("%lf", &tezina);
    if(tezina <= 20)
        printf("Sretan put!\n");
    else{
        double za_izvaditi;
        printf("Unesite koliko cete kilograma izvaditi iz kofera:
        ↪  ");
        scanf("%lf", &za_izvaditi);
        double nova_tezina = tezina - za_izvaditi;
        printf("Nova tezina je %.2f\n", nova_tezina);
        if(nova_tezina <= 20)
            printf("Sretan put\n");
        else
            printf("Kofer je i dalje pretezak!\n");
    }
    return 0;
}
```

Primijetimo da bi bilo pogrešno izostaviti vitičaste zagrade u `else` bloku. Naime, u tom bi se slučaju samo prva naredba (naredba `double za_izvaditi;`) smatrala dijelom `else` bloka, a sve naredbe nakon toga bi se izvodile neovisno o `if` uvjetu.

Također, umjesto uvođenja varijable `nova_tezina`, mogli smo koristiti operator `-=` te imati `tezina -= za_izvaditi`, a zatim dalje raditi s varijablom `tezina` (umjesto `nova_tezina`).

Napomenimo da varijable koje smo deklarirali unutar `if` ili `else` bloka, kao što je primjerice varijabla `za_izvaditi` u gornjem kodu, postoje **samo unutar** toga bloka (izvan tog bloka je ne možemo koristiti).

Zadatak 4.7. Napišite program koji učitava realne brojeve `a`, `b` i `c`. Program ispisuje koliko realnih rješenja ima kvadratna jednadžba

$$ax^2 + bx + c = 0.$$

Pretpostavljamo da je $a \neq 0$.
Upute: koristite diskriminantu $D = b^2 - 4ac$.

Rješenje.

```
1  #include <stdio.h>
2
3  int main(void){
4      double a, b, c;
5      printf("Unesite tri realna broja: ");
6      scanf("%lf %lf %lf", &a, &b, &c);
7      double d = b*b - 4*a*c;
8      if(d < 0)
9          printf("Jednadzba %.2fx^2+%.2fx+%.2f = 0 nema realnih
           ↪  rjesenja\n", a, b, c);
10     else if(d > 0)
11         printf("Jednadzba %.2fx^2+%.2fx+%.2f = 0 ima dva realna
           ↪  rjesenja\n", a, b, c);
12     else // slucaj kad je d == 0
13         printf("Jednadzba %.2fx^2+%.2fx+%.2f = 0 ima jedno realno
           ↪  rjesenje\n", a, b, c);
14     return 0;
15 }
```

Da smo u linijama 10 i 12 imali `if(d > 0)` (dakle bez `else`), odnosno `if(d == 0)`, ispis programa bio bi isti. Razlika je u tome da su to tri razdvojene `if` naredbe i sve tri provjere se izvršavaju (iako samo jedna od njih može biti istinita). U slučaju kad koristimo `if - else if - else`, čim se jedan `if` blok izvrši, ostali se preskoče

Primjer 4.8. Korisnik unosi broj bodova na ispitu iz Uvoda u programiranje. Prema važećim pragovima određuje se ocjena.

```c
#include <stdio.h>

int main(void){
    int bod;
    printf("Unesite broj bodova: ");
    scanf("%d", &bod);
    if(bod >= 90) printf("Ocjena 5.\n");
    else if(bod >= 75) printf("Ocjena 4.\n");
    else if(bod >= 60) printf("Ocjena 3.\n");
    else if(bod >= 45) printf("Ocjena 2.\n");
    return 0;
}
```

Pogrešno bi bilo napisati sljedeće:

```c
if(bod >= 45) printf("Ocjena 2.\n");
else if(bod >= 60) printf("Ocjena 3.\n");
else if(bod >= 75) printf("Ocjena 4.\n");
else if(bod >= 90) printf("Ocjena 5.\n");
```

```
Unesite broj bodova: 78
Ocjena 2.
```

Budući da je prvi `if` zadovoljen, ostali se uvjeti preskoče.
Slično, sljedeći kod ispisuje "previše", budući da se svaki `if` provjerava zasebno.

```c
if(bod >= 45) printf("Ocjena 2.\n");
if(bod >= 60) printf("Ocjena 3.\n");
if(bod >= 75) printf("Ocjena 4.\n");
if(bod >= 90) printf("Ocjena 5.\n");
```

```
Unesite broj bodova: 78
Ocjena 2.
Ocjena 3.
Ocjena 4.
```

Primjer 4.9. Program koji ispisuje veći od dva unesena broja.

```
#include <stdio.h>

int main(void){
    int a, b, max;
    char c;
    printf("Unesite dva broja: ");
    scanf("%d %d", &a, &b);
    max = (a >= b) ? a : b;
    c =  (a >= b) ? 'a' : 'b';
    printf("Veci je %c = %d.", c, max);
    return 0;
}
```

Uvjetne operatore u linijama 8 i 9 mogli smo zamijeniti sljedećom `if else` naredbom:

```
if(a >= b){
    max = a;
    c = 'a';
}
else{
    max = b;
    c = 'b';
}
```

Poglavlje 5

Petlje

Primjer 5.1. Odrediti vrijednost varijable `x` u svakom trenutku izvršavanja sljedećeg programa. Odrediti što će program ispisati.

```c
#include <stdio.h>

int main(void) {
    int x = 100;
    printf("Postfiks:\n");
    printf("x: %d\n", x);
    printf("x++: %d\n", x++); // prvo vrati x, onda digni za 1
    printf("x nakon: %d\n", x);

    x = 100;
    printf("Prefiks:\n");
    printf("x: %d\n", x);
    printf("++x: %d\n", ++x);   // prvo digni za 1 pa vrati x
    printf("x nakon: %d\n", x);

    printf("Dekrement:\n")
    x = 100;
    printf("%d\n", --x);
    x = 100;
    printf("%d\n", x--);

    return 0;
}
```

Ispis gornjeg programa je sljedeći:

```
Postfiks:
x: 100
x++: 100
x nakon: 101
Prefiks:
x: 100
++x: 101
x nakon: 101
Dekrement:
99
100
```

Primjer 5.2. Sljedeći program ispisuje sve znamenke unesenog prirodnog broja krenuvši od znamenke jedinice.

```c
#include <stdio.h>

int main(void){
    int n;
    printf("Unesite prirodni broj: ");
    scanf("%d", &n);
    printf("Znamenke broja %d su:\n", n);
    while(n > 0){
        printf("%d\n", n%10);
        n /= 10;
    }
    return 0;
}
```

Izraz `n%10` daje nam zadnju znamenku broja `n`. Izraz `n/=10` "briše" zadnju znamenku broja `n`. Taj postupak ponaljamo sve dok `n` ne postane jednak 0. Uočimo da je n nakon izvršavanja petlje jednak 0 (ispišite ga). Često ulazne podatke želimo sačuvati, primjerice radi ponovnog korištenja u nastavku programa. To možemo učiniti korištenjem dodatne varijable pomoću koje ćemo spremiti originalnu vrijednost od n.

Zadatak 5.3. Napišite program koji ispisuje sve znamenke unesenog prirodnog broja krenuvši od najznačajnije znamenke.

Rješenje.

```c
#include <stdio.h>

int main(void){
    int n, djelitelj = 1;
    printf("Unesite prirodan broj: ");
    scanf("%d", &n);
    while(n / djelitelj >= 10)
        djelitelj *= 10;
    while(djelitelj > 0){
        printf("%d\n", n/djelitelj);
        n %= djelitelj;
        djelitelj /= 10;
    }
    return 0;
}
```

Primjer 5.4. Program koji unosi cijele brojeve sve dok se ne unese 0 i zatim ispisuje prosjek svih unesenih brojeva (uključujući i 0).

```c
#include <stdio.h>

int main(void) {
    int x, n = 0;
    double sum = 0;
    do {
        printf("Unesite cijeli broj: ");
        scanf("%d", &x);
        sum += x;
        n++;
    } while(x != 0);
    double av = sum/n;
    printf("Prosjek je %lf", av);
    return 0;
}
```

Umjesto `do-while petlje`, mogli smo koristiti `while` petlju u kombinaciji s naredbom `break`. Tada je petlja sljedeća:

```
while(1){
    printf("Unesite cijeli broj: ");
    scanf("%d", &x);
    sum += x;
    n++;
    if(x == 0)
        break;
}
```

Primjer 5.5. Program koji učitava prirodni broj `n` i ispisuje koliko je $n!$. Prisijetimo se, $n! = 1 \cdot 2 \cdot \cdots \cdot (n-1) \cdot n$

```
#include <stdio.h>

int main(void){
    int n;
    long long int fact = 1;
    printf("Unesite broj: ");
    scanf("%d", &n);
    for(int i = 1; i <= n; ++i)
        fact *= i;
    printf("%d! = %ld", n, fact);
    return 0;
}
```

Faktorijele brzo rastu pa je prikladno koristiti varijablu tipa `long`, `unsigned` i sl.
Varijablu `int i` deklarirali smo unutar `for` petlje. Tako deklarirana varijabla nije dostupna van petlje, što i želimo budući da je jedina svrha brojač unutar petlje. Probajte ispisati varijablu `i` nakon izvršavanja petlje. Ako bismo htjeli koristiti varijablu `i` izvan for petlje, morali bismo ju deklarirati izvan `for` petlje.

Primjer 5.6. Sljedeći program provjerava je li broj prost. Prirodni broj $n > 1$ je prost ako su mu jedini djelitelji 1 i n.

```
#include <stdio.h>

```

```
3  int main(void){
4      int n;
5      printf("Unesite prirodan broj: ");
6      scanf("%d", &n);
7      int prost = 1;
8      for(int i = 2; i < n; i++)
9          if(n%i == 0) {
10             prost = 0;
11             break;
12         }
13     if(prost && n > 1) printf("%d je prost", n);
14     else printf("%d nije prost broj", n);
15     return 0;
16 }
```

U petlji je dovoljno provjeriti djeljivost brojevima manjim ili jednakim $\sqrt{n}$, što možemo pisati kao uvjet `i*i <= n`. Naime, ako je broj složen, barem jedan djelitelj mora biti manji ili jednak $\sqrt{n}$.

Zadatak smo mogli riješiti bez korištenja varijable `prost` na sljedeći način:

```
1  #include <stdio.h>
2
3  int main(void){
4      int n, i;
5      printf("Unesite prirodan broj: ");
6      scanf("%d", &n);
7      for(i = 2; i < n; i++)
8          if(n%i == 0)
9              break;
10     if(i == n && n > 1) printf("%d je prost", n);
11     else printf("%d nije prost broj", n);
12     return 0;
13 }
```

Primjer 5.7. Korisnik upisuje brojeve sve dok ne unese 0. Program računa prosjek svih neparnih učitanih brojeva.

```
1  #include <stdio.h>
```

```
2
3  int main(void){
4      int x, sum = 0, n = 0;
5      while(1){
6          printf("Unesite broj: ");
7          scanf("%d", &x);
8          if(x == 0) break;
9          if(x%2 == 0) continue;
10         sum += x;
11         n++;
12     }
13     if(n == 0) printf("Nije ucitan nijedan neparan broj!");
14     else printf("Prosjek je %f", (double)sum/n);
15     return 0;
16 }
```

Budući da su varijable `sum` i `n` tipa `int`, imali bismo cjelobrojno dijeljenje pri računanju prosjeka. To možemo zaobići konverzijom iz tipa `int` u tip `double`.

Primjer 5.8. Za zadani $n > 1$ program ispisuje:

```
1
22
333
...
nnn ...  n
```

```
1  #include <stdio.h>
2
3  int main(void){
4      int n, i, j;
5      printf("Unesite n: ");
6      scanf("%d", &n);
7
8      for(i = 1; i <= n; i++){
9          for(j = 1; j <= i; j++)
10             printf("%d", i);
11         printf("\n");
```

```
    }

    return 0;
}
```

Primjer 5.9. Napišite program koji za zadani $n \geq 2$ ispisuje sve prirodne brojeve $x, y > 1$ za koje vrijedi $x^2 + y^2 = n^2$.

```
#include <stdio.h>

int main(void){
    int n, x, y;
    printf("Unesite n: ");
    scanf("%d", &n);

    for(x = 2; x < n; x++)
        for(y = 2; y < n; y++)
            if(x*x+y*y == n*n) printf("%d^2 + %d^2 = %d^2\n", x, y,
            ↪  n);

    return 0;
}
```

Uočimo da će se ispisati oba para (x, y) i (y, x). Ako želimo izbjeći ponavljanje, možemo proći samo po $y \leq x$ (ili $y \geq x$).

Primjer 5.10. Napišite program koji za unesene prirodne brojeve $a < b$ ispisuje sve prirodne brojeve iz intervala $[a, b]$ koji su potpuni kvadrati. Za x kažemo da je potpuni kvadrat ako postoji y takav da je $x = y^2$.

```
#include <stdio.h>

int main(void){
    int a, b, i, j;
    printf("Unesite a i b: ");
    scanf("%d %d", &a, &b);
    for(i = a; i <= b; i++){
        int potpun = 0;
        for(j = 1; j*j <= i; j++)
```

```
            if(j*j == i){
                potpun = 1;
                break;
            }
        if(potpun) printf("%d ", i);
    }
    return 0;
}
```

Uočimo da se `break` odnosi samo na najbližu petlju u kojoj se nalazi (u ovom slučaju j-petlju).
Analogno vrijedi i za `continue`.

Zadatak 5.11. Napišite program koji za unesene prirodne brojeve $a < b$ ispisuje prvi (najmanji) potpuni kvadrat iz intervala $[a, b]$. Ako takvog nema, treba ispisati poruku.

Rješenje. Treba upotrijebiti drugi put `break` pod uvjetom da je `potpun = 1`. Alternativno, nakon ispisa možemo odmah napraviti `return 0;`.

```c
#include <stdio.h>

int main(void) {
    int a, b;
    printf("Unesite brojeve a i b: ");
    scanf("%d %d", &a, &b);
    int potpun = 0;
    for(int i = a; i <= b; ++i){
        for(int j = 1; j*j <= i; ++j){
            if(j*j == i){ // i je potpuni kvadrat
                potpun = 1;
                break; // odnosi se samo na jednu petlju (po j)
            }
        }
        if(potpun == 1){
            printf("%d je najmanji potpuni kvadrat\n", i);
            break; // odnosi se na petlju po i
        }
    }
    if(potpun == 0) printf("Nema potpunih kvadrata\n");
```

```
21      return 0;
22  }
```

Poglavlje 6

Naredba `switch case`

Naredbu `switch case` možemo koristiti kod višestrukog grananja umjesto `if-else if-else` naredbe kad je uvjet grananja provjera jednakosti

Sintaksa je sljedeća:

```
switch(x) {
     case uvjet1:
          ...
          break;
     case uvjet2:
          ...
          break;
     ...
     default:
          ...
}
```

Primjer 6.1. Napišite program koji učitava dva realna broja te znak za aritmetičku operaciju. Program treba ispisati rezultat zadane operacije.

```
1    #include <stdio.h>
2
3    int main(void){
4        double a, b;
5        char op;
```

```
6           printf("Unesite dva broja: ");
7           scanf("%lf %lf", &a, &b);
8           printf("Unesite aritmeticku operaciju: ");
9           scanf(" %c", &op); // primijetite rrazmak prije %c!
10          switch(op){
11              case '+':
12                  printf("%f %c %f = %f\n", a, op, b, a+b);
13                  break;
14              case '-':
15                  printf("%f %c %f = %f\n", a, op, b, a-b);
16                  break;
17              case '*':
18                  printf("%f %c %f = %f\n", a, op, b, a*b);
19                  break;
20              case '/':
21                  if(b != 0)
22                      printf("%f %c %f = %f\n", a, op, b, a/b);
23                  else
24                      printf("Dijeljenje s 0 nije definirano!\n");
25                  break;
26              default:
27                  printf("Uneseni znak ne predstavlja aritmeticku
                    ↪  operaciju!\n");
28          }
29          return 0;
30      }
```

Primijetite da smo u funkciji `scanf` kod učitavanja znaka aritmetičke operacije (linija 9) stavili razmak ispred `%c`. Razlog za to je sljedeći. Kod učitavanja realnih brojeva (linija 7), na kraju smo stisnuli 'enter', odnosno znak `'\n'`. Taj se znak nije spremio u varijablu nego je ostao u tzv. bufferu (ulaznom spremniku). Stavljanjem razmaka ispred `%c` rekli smo funkciji `scanf` da preskoči sve znakove praznine (razmak, prelazak u novi red) koje su ostali u bufferu nakon prethoodnog unosa. Kada ne bismo stavili razmak prije `%c`, u varijablu `op` spremio bi se znak `'\n'` (koji je ostao u bufferu nakon učitavanja brojeva).

Gornji program mogli smo napisati korištenjem `if` naredbe umjesto `switch case` naredbe. Tada je dio programa koji izvodi zadanu operaciju sljedeći:

```
1       if(op == '+')
```

```
2           printf("%f %c %f = %f\n", a, op, b, a+b);
3       else if(op == '-')
4           printf("%f %c %f = %f\n", a, op, b, a-b);
5       else if(op == '*')
6               printf("%f %c %f = %f\n", a, op, b, a*b);
7       else if(op == '/'){
8           if(b != 0)
9               printf("%f %c %f = %f\n", a, op, b, a/b);
10          else
11              printf("Dijeljenje s 0 nije definirano!\n");
12      }
13      else
14          printf("Uneseni znak ne predstavlja aritmeticku
            ↪ operaciju!\n");
```

Zadatak 6.2. Napišite program koji za učitani prirodni broj n ispisuje je li zadnja znamenka broja 3^n prost broj ili ne.

Rješenje.

```
1       #include <stdio.h>
2
3       int main(void){
4           int n, pot = 1;
5           printf("Unesite prirodan broj: ");
6           scanf("%d", &n);
7           for(int i = 1; i <= n; ++i) pot = (pot * 3) % 10;
8           printf("Zadnja znamenka ");
9           switch(pot % 10){
10              case 2:
11              case 3:
12              case 5:
13              case 7:
14                  printf("je ");
15                  break;
16              default:
17                  printf("nije ");
18          }
19          printf("prost broj\n");
20          return 0;
```

```
21      }
```

Grane koje imaju iste naredbe možemo navesti jednu za drugom, a pripadne naredbe samo u posljednjoj grani. U ovom slučaju, to su grane `case 2`, `case 3`, `case 5` i `case 7`. Naredbe u grani `case 7` odnose se i na sve prijašnje grane.

Poglavlje 7

Razni zadaci

Zadatak 7.1. Napišite program koji učitava prirodni broj `n` te ispisuje sve trojke prirodnih brojeva `a`, `b` i `c` koji su strogo manji od `n` te je $a^3 + b^3 > c^3$.

Rješenje.

```
#include <stdio.h>

int main(void){
    int n;
    printf("Unesite prirodni broj: ");
    scanf("%d", &n);
    for(int a = 1; a < n; ++a)
        for(int b = 1; b < n; ++b)
            for(int c = 1; c < n; ++c)
                if(a*a*a + b*b*b > c*c*c)
                    printf("%d %d %d\n", a, b, c);
    return 0;
}
```

Zadatak 7.2. Napišite program koji unosi cijeli broj `n` te u padajućem poretku (od najvećeg do najmanjeg) ispisuje sve prirodne brojeve strogo manje od `n` koji su relativno prosti s `n`.

Rješenje. Dva su broja relativno prosta ako im je jedini zajednički djelitelj 1.

```
#include <stdio.h>
```

```
2
3  int main(void){
4      int n, i;
5      printf("Unesite cijeli broj: ");
6      scanf("%d", &n);
7      for(i = n - 1; i > 0; --i){
8          //provjera relativne prostosti
9          int d;
10         for(d = 2; d <= i; ++d)
11             if(n%d == 0 && i%d == 0) // nasli smo zajednickog
               ↪  djelitelja
12                 break;
13         if(d == i+1) // znaci da je for petlja dosla do kraja (bez
           ↪  da smo koristili break)
14             printf("%d\n", i);
15     }
16     return 0;
17 }
```

Uočimo da sad nismo mogli varijablu `d` deklarirati unutar `for` petlje jer inače ne bismo mogli napraviti provjeru u liniji 13.

Zadatak 7.3. Program učitava cijele brojeve dok ne učita 0. Treba ispisati onog koji ima najveću apsolutnu vrijednost i koji je po redu. Ako ima više brojeva za koji se postiže maksimum, treba ispisati posljednjeg od njih.

Rješenje.

```
1  #include <stdio.h>
2
3  int main(void){
4      int x, max_ind, i = 0, max_broj, max_aps = 0;
5      while(1){
6          scanf("%d", &x);
7          if(x == 0) break;
8          ++i;
9          int aps = x < 0 ? -x : x;
10         if(aps >= max_aps){
11             max_aps = aps;
12             max_ind = i;
```

```
13              max_broj = x;
14          }
15      }
16      if(i > 0)
17          printf("Broj %d ima najvecu apsolutnu vrijednost, indeks:
            ↪  %d\n", max_broj, max_ind);
18      else
19          printf("Nije unesen nijedan broj razlicit od nule.\n");
20      return 0;
21  }
```

Budući da razmatramo cijele brojeve različite od 0, sigurno znamo da će njihova apsolutna vrijednost biti veća od 0 pa će pri unosu prvog broja `max_aps` biti upravo apsolutna vrijednost prvog učitanog broja.

Zadatak 7.4. Program učitava cijele brojeve dok se ne učita 0. Program treba ispisati najveći učitani cijeli broj (ne računajući zadnje učitani 0) i njegov indeks.

Rješenje. Ne možemo inicijalizirati maksimum na 0 jer su cijeli brojevi neograničeni odozdo, odnosno maksimum može biti manji od 0. Prilikom unosa prvog broja inicijalizirajte maksimum.

```
1   #include <stdio.h>
2
3   int main(void){
4       int x, max_ind = 0, i = 0, max_broj, prvi = 1;
5       while(1){
6           scanf("%d", &x);
7           if(x == 0) break;
8           ++i;
9           if(prvi || x >= max_broj){
10              max_broj = x;
11              max_ind = i;
12              prvi = 0;
13          }
14      }
15      if(i > 0)
16          printf("Broj %d je najveci, indeks: %d\n", max_broj,
            ↪  max_ind);
```

```
17      else
18          printf("Nije unesen nijedan broj razlicit od nule.\n");
19      return 0;
20  }
```

Zadatak 7.5. Napišite program koji za unesene prirodne brojeve $n, k \in \mathbb{N}$ računa $\binom{n}{k}$.

Rješenje. Imamo:

$$\binom{n}{k} = \frac{n!}{k!(n-k)!}$$

Kraćenjem možemo dobiti sljedeća dva izraza:

$$\frac{n \cdot (n-1) \cdot \ldots \cdot (k+1)}{(n-k)!} \quad \text{ili} \quad \frac{n \cdot (n-1) \cdot \ldots \cdot (n-k+1)}{k!}$$

Budući da faktorijele brzo rastu, pogodnije je izabrati onaj s manjim nazivnikom, tj. ako je $n - k < k$ računamo prvi izraz, inače drugi.

```c
#include <stdio.h>

int main(void){
    int n, k;
    unsigned int nazivnik = 1, brojnik = 1;
    printf("Unesite n i k: ");
    scanf("%d %d", &n, &k);
    if(n < 0 || k < 0 || k > n){
        printf("Neispravni podaci.\n");
        return 1;
    }
    int i;
    if(n-k < k){
        for(i=n; i>k; i--) brojnik *= i;
        for(i=1; i<=n-k; i++) nazivnik *= i;
    }else{
        for(i=n; i>n-k; i--) brojnik *= i;
        for(i=1; i<=k; i++) nazivnik *= i;
    }
    printf("%d povrh %d: %u", n, k, brojnik/nazivnik);
    return 0;
}
```

Budući da znamo da je $\binom{n}{k}$ cijeli broj, možemo koristiti cjelobrojno dijeljenje (`brojnik/nazivnik`).
Gornji kod smo mogli kraće napisati ovako:

```
if(n-k < k){
    for(i=1; i<=n-k; i++){
        brojnik *= (n-i+1);
        nazivnik *= i;

    }
}else{
    for(i=1; i<=k; i++){
        brojnik *= (n-i+1);
        nazivnik *= i;

    }
}
```

Alternativno, mogli smo koristiti jednu varijablu `unsigned int binom` i računati na sljedeći način:

```
unsigned int binom = 1;
if(n-k < k){
        for(i=n; i>k; i--) binom *= i;
        for(i=1; i<=n-k; i++) binom /= i;
}else{
        for(i=n; i>n-k; i--) binom *= i;
        for(i=1; i<=k; i++) binom /= i;
}
```

ili

```
unsigned int binom = 1;
if(n-k < k){
    for(i=1; i<=n-k; i++){
        binom *= (n-i+1);
        binom /= i;
       }
}else{
```

```
8       for(i=1; i<=k; i++){
9           binom *= (n-i+1);
10          binom /= i;
11      }
12  }
```

U svakom koraku dijeljenje je egzaktno (zašto?).

Zapis broja u bazi

- Znamenke u bazi b su $\{0, 1, \ldots, b-1\}$
- Ako je $b > 10$, tada neke znamenke prikazujemo velikim slovima
- Primjerice, za $b = 16$, znamenke su $\{0, 1 \ldots, 9, A, B, C, D, E, F\}$
- Zapis broja a u bazi b označavamo s

$$a = (a_n a_{n-1} \ldots a_1 a_0)_b,$$

što možemo raspisati kao

$$a = a_n b^n + a_{n-1} b^{n-1} + \cdots + a_1 b^1 + a_0 b^0 = \sum_{i=0}^{n} a_i b^i$$

- Primjerice, $(20)_{10} = (10100)_2 = (24)_8 = (14)_{16}$
- Prikaz broja a u bazi b dobivamo uzastopnim dijeljenjem a s b te zapisivanjem unatrag svih ostataka pri dijeljenju
- Primjerice, zapis broja 205 u bazi 8 računamo na sljedeći način:

$$\begin{aligned} 205 &= 25 \cdot 8 + 5 \\ 25 &= 3 \cdot 8 + 1 \\ 3 &= 0 \cdot 8 + 3 \end{aligned}$$

Rezultat je $(205)_{10} = (315)_8$

Zadatak 7.6. Napišite program koji unosi prirodne brojeve n i b (pri čemu

je $b \geq 2$). Program treba ispisati koliko znamenaka ima zapis broja n u bazi b.

Rješenje. Kada bismo brojali znamenke u decimalnom zapisu učitanog broja, dijelili bismo broj s 10 („brisali" posljednju znamenku) sve dok ne dođemo do 0 te brojali koliko operacija dijeljenja smo izvršili. Slično radimo i sada, samo umjesto s 10 broj dijelimo s `b`.

```c
#include <stdio.h>

int main(void){
    int n, b;
    printf("Unesite prirodne brojeve: ");
    scanf("%d %d", &n, &b);
    int n_kopija = n, br = 0;
    while(n > 0){
        ++br;
        n /= b;
    }
    printf("Broj %d u bazi %d ima %d znamenaka\n", n_kopija, b, br);
    return 0;
}
```

Zadatak 7.7. Napišite program koji učitava prirodni broj `b` (`b` ≥ 2). Zatim program učitava prirodne brojeve sve dok se ne unese 0. Za svaki uneseni broj (osim 0) treba ispisati zbroj njegovih znamenaka u bazi `b`.

Rješenje.

```c
#include <stdio.h>

int main(void){
    int b;
    printf("Unesite bazu: ");
    scanf("%d", &b);
    int n;
    while(1){
        printf("Unesite prirodni broj: ");
        scanf("%d", &n);
        if(!n) break;
```

```
12          int zbroj = 0, n_kopija = n;
13          while(n > 0){
14              zbroj += n%b;
15              n /= b;
16          }
17          printf("Zbroj znamenaka broja %d u bazi %d je %d\n",
            ↪  n_kopija, b, zbroj);
18      }
19      return 0;
20  }
```

Zadatak 7.8. Napišite program koji učitava prirodan broj `n` i prirodan broj `b` (`b` ≥ 2). Program ispisuje duljinu najduljeg uzastopnog niza istih znamenaka u zapisu broja `n` u bazi `b`.
Primjerice, za uneseni broj $n = 221$ i $b = 2$, program ispisuje 3 jer je $(221)_{10} = (11011101)_2$, a najdulji niz uzastopnih istih znamenaka je 111.

Rješenje.

```
1   #include <stdio.h>
2
3   int main(void){
4       int n, b, max_dulj = 0, max_znam;
5       printf("Unesite prirodni broj i bazu: ");
6       scanf("%d %d", &n, &b);
7
8       while(n > 0){
9           int tren_znam = n%b;
10          int dulj = 1;
11          n /= b;
12          while(n%b == tren_znam){
13              n/=b;
14              ++dulj;
15          }
16          if(dulj >= max_dulj){
17              max_znam = tren_znam;
18              max_dulj = dulj;
19          }
20      }
21      printf("Najdulji niz uzastopnih istih znamenki je duljine %d, a
        ↪  radi se o znamenci %d\n", max_dulj, max_znam);
```

```
22      return 0;
23  }
```

Zadatak 7.9. Napišite program koji unosi prirodan broj `n` te pomoću znaka `*` crta

a) Znak `X` širine i visine `2n+1`. Primjerice, za `n = 2`, program crta

```
1  *   *
2   * *
3    *
4   * *
5  *   *
```

b) Ispunjeni trokut visine `n`. Primjerice, za `n = 5`, program crta

```
1      *
2     ***
3    *****
4   *******
5  *********
```

c) Ispunjeni dijamant visine `2n+1` Primjerice, za `n = 3`, program crta

```
1     *
2    ***
3   *****
4  *******
5   *****
6    ***
7     *
```

Rješenje. a)

```
1  #include <stdio.h>
2
```

```
int main(void){
    int n;
    printf("Unesite prirodan broj: ");
    scanf("%d", &n);
    for(int i = 0; i < 2*n+1; ++i){
        for(int j = 0; j < 2*n+1; ++j){
            if(j == i || j == 2*n-i)
                printf("*");
            else
                printf(" ");
        }
        printf("\n");
    }
    return 0;
}
```

b)

```
int main(void){
    int n;
    printf("Unesite prirodan broj: ");
    scanf("%d", &n);
    for(int i = 1; i <= n; ++i){
        for(int j = 1; j <= 2*n; ++j){
            if(j >= n+1-i && j <= n-1+i)
                printf("*");
            else
                printf(" ");
        }
        printf("\n");
    }
    return 0;
}
```

c) Najlakše je ovaj zadatak riješiti na način da prvo nacrtamo jedan trokut visine $n+1$ (to znamo iz b) dijela zadatka), a zatim nacrtamo obrnuti trokut visine n.

```
#include <stdio.h>
```

```
2
3  int main(void){
4      int n;
5      printf("Unesite prirodan broj: ");
6      scanf("%d", &n);
7      for(int i = 1; i <= n+1; ++i){ // trokut visine n+1
8          for(int j = 1; j <= 2*(n+1); ++j){
9              if(j >= n+2-i && j <= n+i)
10                 printf("*");
11             else
12                 printf(" ");
13         }
14         printf("\n");
15     }
16
17     for(int i = n; i >= 1; --i){ // obrnuti trokut visine n
18         for(int j = 1; j <= 2*n; ++j){
19             if(j >= n+1-(i-1) && j <= n+i)
20                 printf("*");
21             else
22                 printf(" ");
23         }
24         printf("\n");
25     }
26     re
```

Zadatak 7.10. Napišite program koji učitava prirodni broj `n` i ispisuje broj prijelaza između parne i neparne znamenke u njegovom decimalnom zapisu. Primjerice, za $n = 238245$ imamo 3 prijelaza: prijelaz sa 2 na 3, sa 3 na 8 i sa 4 na 5.

Rješenje.

```
1  #include <stdio.h>
2
3  int main(void){
4      int n;
5      printf("Unesite prirodan broj: ");
6      scanf("%d", &n);
7      int preth = n % 10, broj = 0;
```

```
    n /= 10;
    while(n > 0){
        int tren = n % 10;
        if(tren % 2 != preth % 2) ++broj;
        preth = tren;
        n /= 10;
    }
    printf("%d\n", broj);
    return 0;
}
```

Zadatak 7.11. Napišite program koji učitava prirodni broj `n` i ispisuje koliko njegovih znamenaka je strogo veće od obje susjedne znamenke. Prva i posljednja znamenka ne mogu se brojiti. Primjerice, za $n = 3527481$ rezultat je 3 (znamenke 5, 7 i 8).

Rješenje.

```
#include <stdio.h>

int main(void){
    int n;
    printf("Unesite prirodan broj: ");
    scanf("%d", &n);
    int broj = 0;
    int desna = n % 10;
    n /= 10;
    if(n > 0){
        int srednja = n % 10;
        n /= 10;
        while(n > 0){
            int lijeva = n % 10;
            if(srednja > lijeva && srednja > desna) ++broj;
            desna = srednja;
            srednja = lijeva;
            n /= 10;
        }
    }
    printf("%d\n", broj);
    return 0;
```

```
}
```

Zadatak 7.12. Napišite program koji učitava prirodni broj `n` i ispisuje koliko različitih znamenaka sadrži njegov decimalni zapis. Primjerice, broj 303155 sadrži četiri različite znamenke.

Rješenje.

```c
#include <stdio.h>

int main(void){
    int n;
    printf("Unesite prirodan broj: ");
    scanf("%d", &n);
    int broj = 0;
    for(int z = 0; z <= 9; ++z){
        int kopija = n;
        do{
            if(kopija % 10 == z) {
                ++broj;
                break;
            }
            kopija /= 10;
        }while(kopija > 0);
    }
    printf("Broj %d sadrzi %d razlicitih znamenaka\n", n, broj);
    return 0;
}
```

Zadatak 7.13. Napišite program koji učitava prirodni broj `n` i ispisuje je li njegov decimalni zapis palindrom, odnosno čita li se jednako slijeva nadesno i zdesna nalijevo. Primjerice, 12321 jest palindrom, a 12341 nije.

Rješenje.

```c
#include <stdio.h>

int main(void){
    int n, kopija, obrnuti = 0;
```

```
5       printf("Unesite prirodan broj: ");
6       scanf("%d", &n);
7       kopija = n;
8       do{
9           obrnuti = 10*obrnuti + kopija % 10;
10          kopija /= 10;
11      }while(kopija > 0);
12      printf("%d %s palindrom\n", n, n == obrnuti ? "je" : "nije");
13      return 0;
14  }
```

Zadatak 7.14. Napišite program koji učitava prirodni broj `n` i ispisuje njegov najveći prosti djelitelj. Primjerice, za $n = 84$ rezultat je 7. Pretpostavite da je $n \geq 2$.

Rješenje.

```
1   #include <stdio.h>
2
3   int main(void){
4       int n, najveci = 1;
5       printf("Unesite prirodan broj: ");
6       scanf("%d", &n);
7       for(int d = 2; d <= n / d; ++d){
8           while(n % d == 0){
9               najveci = d;
10              n /= d;
11          }
12      }
13      if(n > 1) najveci = n;
14      printf("Najveci prosti djelitelj od %d je %d\n", najveci);
15      return 0;
16  }
```

Zadatak 7.15. Napišite program koji učitava cijele brojeve sve do unosa nule, koja nije dio niza. Program ispisuje duljinu najduljeg uzastopnog strogo rastućeg dijela unesenog niza. Ako je prvi uneseni broj nula, ispišite 0. Primjerice, za unos $3, 5, 7, 2, 4, 6, 8, 1, 0$ rezultat je 4 jer je najdulji strogo rastući podniz $2, 4, 6, 8$.

Rješenje.

```c
#include <stdio.h>

int main(void){
    int x, preth;
    printf("Unesite cijeli broj: ");
    scanf("%d", &x);
    if(x == 0){
        printf("0\n");
        return 0;
    }
    preth = x;
    int tren_duljina = 1, max_duljina = 1;
    while(x != 0){
        printf("Unesite cijeli broj: ");
        scanf("%d", &x);
        if(x > preth) ++tren_duljina;
        else tren_duljina = 1;
        if(tren_duljina > max_duljina)
            max_duljina = tren_duljina;
        preth = x;
    }
    printf("Duljina najduljeg strogo rastuceg podniza je %d\n",
    ↪  max_duljina);
    return 0;
}
```

Zadatak 7.16. Napišite program koji učitava cijele brojeve sve do unosa nule, koja nije dio niza. Program ispisuje broj promjena predznaka između dvaju uzastopnih učitanih brojeva. Primjerice, za unos $2, -3, -5, 7, -1, 0$ rezultat je 3.

Rješenje.

```c
#include <stdio.h>

int main(void){
    int preth, x, broj = 0;
    printf("Unesite cijeli broj: ");
```

```
6       scanf("%d", &preth);
7       if( preth == 0){
8           printf("0\n"); return 0;
9       }
10      while(x != 0){
11          printf("Unesite cijeli broj: ");
12          scanf("%d", &x);
13          if((preth < 0 && x > 0) || (preth > 0 && x < 0))
14              ++broj;
15          preth = x;
16      }
17      printf("Broj promjena predznaka u nizu je %d\n", broj);
18      return 0;
19  }
```

Poglavlje 8

Nizovi

Primjer 8.1. Napišite program koji deklarira niz duljine 10 u koji spremamo cijele brojeve. Program zatim učitava 10 cijelih brojeva te ih sprema u niz. Nakon toga je potrebno brojeve ispisati u obrnutom redoslijedu.

```c
#include <stdio.h>

int main(void){
    int a[10]; // deklaracija niza
    for(int i = 0; i < 10; ++i){
        printf("Ucitajte niz[%d]: ", i);
        scanf("%d", &a[i]);
    }
    for(int i = 9; i >= 0; --i)
        printf("%d ", a[i]);
    return 0;
}
```

Primjer 8.2. Što ispisuje sljedeći kod?

```c
#include <stdio.h>

int main(void){
    int n = 4, a[4], i = 0;
    for(i = 0; i < n; ++i)
        a[i] = 2*i;
    for(i = 0; i < n-1; ++i){
```

```
8           printf("%d ", a[++i]);
9           printf("%d\n", a[i--]);
10      }
11      return 0;
12  }
```

Ispis je sljedeći:

```
1  2 2
2  4 4
3  6 6
```

Zadatak 8.3. Napišite program koji učitava prirodni broj `n` te ispisuje sve njegove znamenke, krenuvši s najznačajnijom znamenkom. Nakon toga, za svaku znamenku ispisuje koliko se puta javlja.

Rješenje. Znamenke učitanog broja dohvaćamo na isti način kao u primjeru 5.2 (gledamo ostatak pri cjelobrojnom dijeljenju s 10 kako bismo dohvatili zadnju znamenku, a zatim broj dijelimo s 10 kako bismo maknuli zadnju znamenku). Ovdje ćemo sve znamenke učitanog broja spremiti u niz kako bismo ih lako mogli ispisati u zadanom poretku. Znamenku jedinica spremamo na indeks `0`, znamenku desetina na indeks `1`, i tako dalje, sve do najznačajnije znamenke koju spremamo na indeks `l-1`, gdje je `l` broj znamenaka učitanog broja. Da bismo ispisali znamenke u zadanom poretku, potrebno je niz u koji smo spremali znamenke ispisati u obrnutom poretku (od zadnjeg elementa do nultog).
Broj pojava znamenke `z` u učitanom brojku pamtimo u nizu `znam` na indeksu `z`. Sve elemente niza `znam` inicijalizirali smo na 0 kako bismo ih u petlji mogli inkrementirati.

```
1  #include <stdio.h>
2
3  int main(void){
4      int n;
5      printf("Unesite prirodni broj: ");
6      scanf("%d", &n);
7      int a[100], znam[10] = {0}, i = 0;
8      while(n > 0){
```

```
        ++znam[n%10];
        a[i++] = n%10;
        n /= 10;
    }
    printf("Znamenke ucitanog broja su:\n");
    for(--i; i >= 0; --i)
        printf("%d ", a[i]);
    printf("\n");
    for(i = 0; i < 10; ++i)
        printf("Znamenka %d javlja se %d puta\n", i, znam[i]);
    return 0;
}
```

Zadatak 8.4. Riješite prethodni zadatak tako da prvo prebrojite koliko znamenaka ima učitani broj, a zatim deklarirate niz čija je duljina jednaka broju znamenaka.

Rješenje.

```
#include <stdio.h>

int main(void){
    int n;
    printf("Unesite prirodni broj: ");
    scanf("%d", &n);
    int kopija_n = n, br_znam = 0;
    while(kopija_n > 0){
        ++br_znam;
        kopija_n/=10;
    }
    int a[br_znam], znam[10] = {0}, i = 0;
    while(n > 0){
        ++znam[n%10];
        a[i++] = n%10;
        n /= 10;
    }
    printf("Znamenke ucitanog broja su:\n");
    for(--i; i >= 0; --i)
        printf("%d ", a[i]);
    printf("\n");
```

```
22      for(i = 0; i < 10; ++i)
23          printf("Znamenka %d javlja se %d puta\n", i, znam[i]);
24      return 0;
25  }
```

Zadatak 8.5. Napišite program koji učitava prirodni broj `n`, a zatim `n` cijelih brojeva. Program ispisuje onaj broj koji je najudaljeniji od svojeg sljedbenika te njegov redni broj pri učitavanju (indeks). Udaljenost se računa kao apsolutna vrijednost razlike.

Rješenje.

```
1   #include <stdio.h>
2
3   int main(void){
4       int n, max_razlika = 0, max_ind;
5       printf("Unesite prirodan broj: ");
6       scanf("%d", &n);
7       int a[n];
8       for(int i = 0; i < n; ++i){
9           printf("Unesite cijeli broj: ");
10          scanf("%d", &a[i]);
11      }
12      for(int i = 0; i < n-1; ++i){
13          int razlika = a[i] - a[i+1];
14          if(razlika < 0) razlika = -razlika;
15          if(razlika >= max_razlika){
16              max_razlika = razlika;
17              max_ind = i;
18          }
19      }
20      printf("Indeks: %d, vrijednost: %d\n", max_ind, a[max_ind]);
21      return 0;
22  }
```

Zadatak 8.6. Napišite program koji učitava cijele brojeve sve dok se ne učita 15 brojeva ili broj 0 te ih sprema u niz (0 se isto sprema u niz). Iz niza je potrebno izbrisati prvi neparan broj.

Rješenje. Element brišemo iz niza tako da sve elemente nakon njega pomaknemo jedno mjesto ulijevo i time pregazimo element. Također je potrebno

dekrementirati varijablu u kojoj pamtimo broj elemenata niza (ako takva postoji).

```c
#include <stdio.h>

int main(void){
    int a[15], n = 0;
    while(n < 15){
        printf("Unesite cijeli broj: ");
        scanf("%d", &a[n++]);
        if(a[n-1] == 0) break;
    }
    for(int i = 0; i < n; ++i){
        if(a[i]%2 == 1){
            for(int j = i; j < n-1; ++j){
                a[j] = a[j+1];
            }
            --n;
            break;
        }
    }
    for(int i = 0; i < n; ++i)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Zadatak 8.7. Napišite program koji učitava cijele brojeve sve dok se ne učita 15 brojeva ili broj 0 te ih sprema u niz (0 se isto sprema u niz). Zatim se učitava cijeli broj `n` te se iz niza brišu svi brojevi djeljivi s `n`.

Rješenje. Zadatak smo mogli riješiti uzastopnom primjenom rješenja prethodnog zadatka, no postoji efikasnije rješenje. Umjesto da svaki element koji moramo obrisati pregazimo kao što smo to radili u prethodnom zadatku, proći ćemo po nizu te elemente koje ne želimo obrisati jednostavnoi prepisati na novo mjesto. U donjem kodu, varijabla `i` u `for` petlji prolazi po nizu, dok varijabla `j` označava poziciju na koju ćemo spremati elemente koji ostaju u nizu (elementi koji nisu djeljivi s `n`).

```c
#include<stdio.h>
```

```
2
3  int main(void){
4      int a[15], l = 0, n;
5      while(l < 15){
6          printf("Unesite cijeli broj: ");
7          scanf("%d", &a[l++]);
8          if(a[l-1] == 0) break;
9      }
10     printf("Unesite prirodni broj: ");
11     scanf("%d", &n);
12     int j = 0;
13     for(int i = 0; i < l; ++i){
14         if(a[i]%n != 0){
15             if(i > j) a[j] = a[i];
16             ++j;
17         }
18     }
19     l = j;
20     for(int i = 0; i < l; ++i)
21         printf("%d ", a[i]);
22     printf("\n");
23     return 0;
24 }
```

Zadatak 8.8. Napišite program koji učitava cijele brojeve sve dok se ne učita 15 brojeva ili broj 0 te ih sprema u niz (0 se ne sprema) tako da je niz cijelo vrijeme sortiran uzlazno.

Rješenje.
Element dodajemo u niz na indeks `i` tako da sve elemente od indeksa `i` do zadnjeg indeksa pomaknemo za jedno mjesto udesno, **krenuvši sa zadnjim elementom niza**.

```
1  #include <stdio.h>
2
3  int main(void){
4      int a[15], n = 0, x;
5      while(n < 15){
6          printf("Unesite cijeli broj: ");
7          scanf("%d", &x);
```

```
8           if(x == 0) break;
9           if(n == 0){
10              a[n++] = x;
11              continue;
12          }
13          if(x <= a[0]){
14              for(int i = n-1; i >= 0; --i)
15                  a[i+1] = a[i];
16              a[0] = x;
17          }
18          else if(x >= a[n-1]){
19              a[n] = x;
20          }
21          else{
22              for(int i = 0; i < n; ++i){
23                  if(x >= a[i] && x <= a[i+1]){
24                      for(int j = n-1; j >= i+1; --j)
25                          a[j+1] = a[j];
26                      a[i+1] = x;
27                      break;
28                  }
29              }
30          }
31          ++n;
32      }
33      for(int i = 0; i < n; ++i)
34          printf("%d ", a[i]);
35      printf("\n");
36      return 0;
37  }
```

Zadatak 8.9. Napišite program koji čita cijele brojeve i sprema ih u niz sve dok se ne učita 15 brojeva ili broj 0 (koji nije dio niza). Program u učitani niz cijelih brojeva dodaje broj $2x + 1$ ispred svakog parnog broja x.

Rješenje. Zadatak bismo mogli riješiti višestrukom primjenom rješenja prošlog zadatka, no postoji efikasniji način. Nakon što smo učitali sve brojeve i spremili ih u niz, prolazimo kroz niz te brojimo koliko ima parnih brojeva. Time smo odredili kolika će biti duljina niza nakon što ispred svakog parnog broja x dodamo `2x+1`.

Nakon toga, varijabla `i` čita elemente niza, krenuvši od zadnjeg elementa niza, dok varijabla `j` označava nove pozicije, krenuvši od pozicije `novi_n - 1` (jer je to zadnja pozicija u nisu nakon dodavanja novih elemenata). Postojeće elemente niza prepisujemo na njihova nova mjesta, a ispred svakog parnog broja dodajemo novi element.

```c
#include <stdio.h>

int main(void){
    int a[30], n = 0, x;
    while(n < 15){
        printf("Unesite cijeli broj: ");
        scanf("%d", &x);
        if(!x) break;
        a[n++] = x;
    }
    int novi_n = n;
    for(int i = 0; i < n; ++i)
        if(a[i]%2 == 0) ++novi_n;

    int j = novi_n-1;
    for(int i = n-1; i >= 0; --i){
        a[j] = a[i];
        --j;
        if(a[i]%2 == 0){
            a[j] = 2*a[i]+1;
            --j;
        }
    }
    for(int i = 0; i < novi_n; ++i)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Zadatak 8.10. Napišite program koji čita cijele brojeve i sprema ih u niz sve dok se ne učita 15 brojeva ili broj 0 (koji nije dio niza). Program u učitani niz cijelih brojeva dodaje broj $2x + 1$ iza svakog parnog broja x.

Rješenje.

```c
#include <stdio.h>

int main(void){
    int a[30], n = 0, x;
    while(n < 15){
        printf("Unesite cijeli broj: ");
        scanf("%d", &x);
        if(!x) break;
        a[n++] = x;
    }
    int novi_n = n;
    for(int i = 0; i < n; ++i)
        if(a[i]%2 == 0) ++novi_n;

    int j = novi_n-1;
    for(int i = n-1; i >= 0; --i){
        if(a[i]%2 == 0){
            a[j] = 2*a[i]+1;
            --j;
        }
        a[j] = a[i];
        --j;

    }
    for(int i = 0; i < novi_n; ++i)
        printf("%d ", a[i]);
    printf("\n");
    return 0;
}
```

Zadatak 8.11. Napišite program koji učitava prirodne brojeve `n` i `m`. Zatim učitava dvije matrice realnih brojeva dimenzija $n \times m$ te ispisuje njihov zbroj.

Rješenje. Matricu učitavamo po retcima (prvo učitamo sve elemente nultog reda, zatim sve elemente prvog reda itd.). Stoga imamo dvije `for` petlje: petlja po `i` prolazi po retcima, dok petlja po `j` prolazi po stupcima.
Matrice zbrajamo "po komponentama". Formalnije, zbroj matrica A i B dimenzija $n \times m$ je matrica C dimenzija $n \times m$ takva da je $C[i][j] = A[i][j] + B[i][j]$.

```c
#include <stdio.h>

int main(void){
    int n, m;
    printf("Unesite dva prirodna broja: ");
    scanf("%d %d", &n, &m);
    double a[n][m], b[n][m];
    for(int i = 0; i < n; ++i){
        for(int j = 0; j < m; ++j){
            printf("Unesite a[%d][%d]: ", i, j);
            scanf("%lf", &a[i][j]);
        }
    }

    for(int i = 0; i < n; ++i){
        for(int j = 0; j < m; ++j){
            printf("Unesite b[%d][%d]: ", i, j);
            scanf("%lf", &b[i][j]);
        }
    }

    for(int i = 0; i < n; ++i){
        for(int j = 0; j < m; ++j){
            printf("%-6.2f ", a[i][j]+b[i][j]);
        }
        printf("\n");
    }
    printf("\n");
    return 0;
}
```

Zadatak 8.12. Napišite program koji učitava prirodne brojeve `n` i `m`. Zatim učitava matricu cijelih brojeva dimenzije $n \times m$. Program ispisuje umnožak ne-nul elemenata u svakom stupcu.

Rješenje. Nakon što učitamo matricu, moramo proći po svim stupcima te izračunati umnožak svih ne-nul elemenata. Vanjska `for` petlja prolazi po stupcima i zato ide do `m`. Unutarnja `for` petlja ide po retcima i zato ide do `n`.

```c
#include <stdio.h>

int main(void){
    int n, m;
    printf("Unesite dva prirodna broja: ");
    scanf("%d %d", &n, &m);
    int a[n][m];
    for(int i = 0; i < n; ++i){
        for(int j = 0; j < m; ++j){
            printf("Unesite a[%d][%d]: ", i, j);
            scanf("%d", &a[i][j]);
        }
    }

    for(int j = 0; j < m; ++j){
        int prod = 1;
        for(int i = 0; i < n; ++i){
            if(a[i][j]) prod *= a[i][j];
        }
        printf("Umnozak ne-nul elemenata u stupcu %d je %d\n", j,
        ↪ prod);
    }

    return 0;
}
```

Zadatak 8.13. Napišite program koji učitava prirodni broj `n`. Zatim učitava kvadratnu matricu cijelih brojeva dimenzije $n \times n$. Program ispisuje sumu elemenata na glavnoj i na sporednoj dijagonali.

Rješenje. Elementi na glavnoj dijagonali nalaze se na poziciji (i, i), dok se elementi na sporednoj dijagonali nalaze na poziciji $(i, n-1-i)$. Stoga možemo sume izračunati u jednoj `for` petlji.

```c
#include <stdio.h>

int main(void){
    int n;
    printf("Unesite prirodni broj: ");
```

```
6       scanf("%d", &n);
7       int a[n][n];
8       for(int i = 0; i < n; ++i){
9           for(int j = 0; j < n; ++j){
10              printf("Unesite a[%d][%d]: ", i, j);
11              scanf("%d", &a[i][j]);
12          }
13
14      int sum_glavna = 0, sum_sporedna = 0;
15      for(int i = 0; i < n; ++i){
16          sum_glavna += a[i][i];
17          sum_sporedna += a[i][n-1-i];
18      }
19      printf("Suma elemenata na glavnoj dijagonali je %d, a suma
        ↪  elemenata na sporednoj dijagonali je %d\n", sum_glavna,
        ↪  sum_sporedna);
20      return 0;
21  }
```

Zadatak 8.14. Napišite program koji učitava niz cjelobrojnih kvadratnih matrica dimenzija $n \times n$ dok se ne učita 10 matrica ili nul-matrica (koja je dio niza). Program ispisuje matrice koje imaju najmanju međusobnu udaljenost, a nalaze se na različitim indeksima.
Udaljenost matrica A i B računa se formulom

$$d(A,B) = \sqrt{\sum_{i=0}^{n-1}\sum_{j=0}^{n-1}(A[i][j] - B[i][j])^2}.$$

Rješenje. Učitane matrice spremamo u trodimenzionalni niz `a`. Zatim računamo udaljenost između svake dvije matrice u nizu. Zbog toga nam trebaju dvije `for` petlje.
Primijetimo da je $dist(A,B) = dist(B,A)$. Da bismo izbjegli duplo računanje te da bismo izbjegli računanje udaljenost matrice od same sebe (što uvijek iznosi 0), unutarnja `for` petlja ide do `i`. Dakle, uvijek računamo udaljenost matrica `a[i]` i `a[j]` takvih da je `j < i`.
Za računanje korijena koristili smo funkciju `sqrt` iz zaglavlja `math.h` koje smo uključili na početku programa.
Budući da tražimo najmanju udaljenost, varijablu `min_dist` ne možemo odmah inicijalizirati jer udaljenost može biti proizvoljno velika. Stoga u

`min_dist` prvo spremamo udaljenost prve dvije matrice u nizu.
Kako bismo na kraju mogli ispisati matrice, moramo pamtiti indekse matrica čija međusobna udaljenost je najmanja. Te indekse pamtimo u varijablama `min_i` i `min_j`.

```
#include <stdio.h>
#include <math.h>

int main(void){
    int a[10][3][3];
    int n = 0, prvi = 1;
    double min_dist;
    int min_i, min_j;
    while(n < 10){
        int nul_matrica = 1;
        for(int i = 0; i < 3; ++i){
            for(int j = 0; j < 3; ++j){
                printf("Unesite element na poziciji (%d, %d): ", i,
                ↪  j);
                scanf("%d", &a[n][i][j]);
                if(a[n][i][j]) nul_matrica = 0;
            }
        }
        ++n;
        if(nul_matrica)
            break;
    }
    for(int i = 0; i < n; ++i){
        for(int j = 0; j < i; ++j){
            double dist = 0;
            for(int k = 0; k < 3; ++k){
                for(int l = 0; l < 3; ++l){
                    dist += (a[i][k][l] - a[j][k][l])*(a[i][k][l] -
                    ↪  a[j][k][l]);
                }
            }
            dist = sqrt(dist);
            if(prvi || dist <= min_dist){
                prvi = 0;
                min_dist = dist;
```

```
34                  min_i = i;
35                  min_j = j;
36              }
37          }
38      }
39      if(n < 2){
40          printf("Nije uneseno dovoljno matrica!\n");
41          return 0;
42      }
43      printf("Najmanja udaljenost je %.2f izmedju matrica na indeksima
        ↪  %d i %d\n", min_dist, min_i, min_j);
44      printf("To su sljedece matrice:\n");
45      for(int i = 0; i < 3; ++i){
46          for(int j = 0; j < 3; ++j){
47              printf("%5d ", a[min_i][i][j]);
48          }
49          printf("\n");
50      }
51      printf("\n");
52      for(int i = 0; i < 3; ++i){
53          for(int j = 0; j < 3; ++j){
54              printf("%5d ", a[min_j][i][j]);
55          }
56          printf("\n");
57      }
58      printf("\n");
59      return 0;
60  }
```